Test case: AL-131

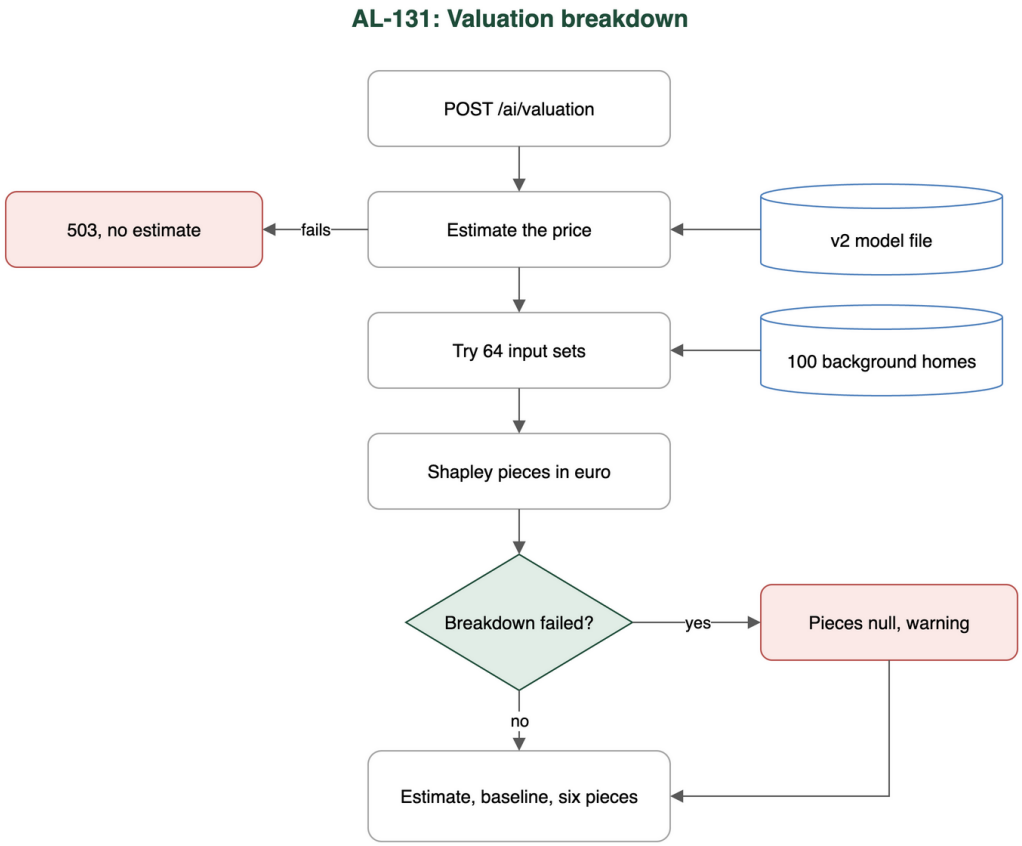
As a data scientist, the valuation endpoint returns how much each house feature added to or took off the estimate

Item	Detail
Story	AL-131: As a data scientist, the valuation endpoint returns how much each house feature added to or took off the estimate
Epic	AL-14 Explain the valuation
Owner	Nic (al-ai: the valuation breakdown)
Sprint	Sprint 2 (27 Oct to 15 Nov 2026)
Wireframe	`wireframe.png` in this folder. AL-131 has no screen, so it is a flow diagram.
Code	`al-ai/app/valuation_explain.py`, `al-ai/app/valuation_endpoint.py`, `al-ai/models/valuation/` (model, background sample, model card) and `al-ai/tests/`, on `main` at commit `d1d1014`
Tested	13 Nov 2026, Nicholas Groenewald

What this tests

Every valuation comes back with a baseline and one euro piece for each of the six inputs. These tests check that the pieces are exact Shapley values, that they add up to the estimate, that the breakdown is fast enough, and that the model card explains how to read it.

Wireframe



Preconditions and test data

- Run `uv sync` once in `al-ai`. No network and no API keys are needed.
- The saved model is in `al-ai/models/valuation/`: the model, `manifest.json`, `background-sample.csv` (100 training homes), `reference-breakdown.json` and `MODEL_CARD.md`.
- The tests use made-up homes written in the test files.
- Only TC-131-02 needs the raw dataset `al-ai/data/raw/daft_housing_data.csv` (not in git) and `AL_FULL_TEST_SPLIT=1`.

Test cases

ID	Acceptance criterion	Steps	Expected result	Result and evidence
TC-131-01	Each estimate also returns a baseline (the average estimate over a fixed sample of training homes) and one euro piece for each of the six inputs: county, floor area, bedrooms, property type, BER and Eircode area.	1. Send a home to `/ai/valuation`. 2. Read `baseline_eur` and `contributions`.	The same baseline for every home, and six pieces, biggest first. Only the routing key shows, never the full Eircode.	Pass. `test_valuation_breakdown.py`: `test_the_response_has_a_baseline_and_one_piece_for_each_of_the_six_inputs`, `test_the_baseline_is_the_same_for_every_home`, `test_the_pieces_are_sorted_by_size_biggest_first`, `test_the_full_eircode_never_appears_in_the_breakdown`.
TC-131-02	For every home in the final test split, the baseline plus the pieces equals the estimate to within EUR 1 (checked by a test).	1. Run the full-split test with `AL_FULL_TEST_SPLIT=1`.	The biggest gap over all 1,630 test homes is under EUR 1.	Pass. `test_valuation_explain.py`: `test_contributions_add_up_to_the_estimate_for_every_property_in_the_final_test_split` (1 passed, 8 min 46 s). Also on fixed homes: `test_the_saved_model_contributions_add_up_to_the_estimate_within_one_euro`.
TC-131-03	The pieces are exact Shapley values worked out on the euro estimate.	1. Run the Shapley code on toy models where I can work out the answer by hand. 2. Run it on the saved model.	The toy answers match my hand working, an input the model ignores gets exactly 0, and the pieces add up to the model's euro answer.	Pass. `test_valuation_explain.py`: `test_a_linear_model_gives_each_input_its_own_step_away_from_the_background_average`, `test_two_inputs_that_interact_share_the_credit_equally`, `test_an_input_the_model_ignores_gets_exactly_zero`, `test_baseline_plus_contributions_is_the_model_answer_for_the_home`.
TC-131-04	Adding the breakdown adds no more than 500 ms to a `/valuation` call on a team laptop.	1. Load the model and warm it up. 2. Time the estimate alone and the breakdown, 15 calls each.	The breakdown adds under 500 ms.	Pass. Timed by hand, with no automated test: breakdown median 191 ms, worst 208 ms. Estimate alone median 14 ms, worst 17 ms. Apple M5 Pro, 13 Nov 2026. The model card says median 196 ms, worst 230 ms (27 Oct).
TC-131-05	The model card explains how to read the	1. Read "How to read the	The card explains the baseline, says	Partly met (Note 1). The card explains the baseline and says a

ID	Acceptance criterion	Steps	Expected result	Result and evidence
	numbers: how far each input moved this estimate from the average, not what changing the house would do.	breakdown" in `al-ai/models/valuation/MODEL_CARD.md`.	each piece is how far that input moved this estimate, and says it is not a what-if.	piece is "how much that input moved the estimate away from the baseline for this home". It never says a piece is not what changing the house would do.

Test run

From `al-ai` on `main` at `d1d1014`, 13 Nov 2026, Apple M5 Pro:

```
uv run pytest -q -rs -p no:cacheprovider tests/test_valuation_explain.py tests/test_valuation_breakdown.py tests/test_valuation_breakdown_failure.py tests/test_docker_image_files.py
```

Result: 53 passed, 1 skipped. The skipped test is the full-split test, which only runs with `AL_FULL_TEST_SPLIT=1`.

```
AL_FULL_TEST_SPLIT=1 uv run pytest -q -rs -p no:cacheprovider "tests/test_valuation_explain.py::test_contributions_add_up_to_the_estimate_for_every_property_in_the_final_test_split"
```

Result: 1 passed in 8 min 46 s.

Notes

1. TC-131-05: one more sentence in the model card, saying a piece is not what changing the house (for example adding a bedroom) would do, would fully meet this.
2. TC-131-02: on a fresh clone the full-split test skips, because the raw dataset is not in git. For this run I copied the dataset in.
3. TC-131-04: I timed the functions directly, not a full HTTP call, so it measures what the breakdown costs and leaves out the network.
4. `test_docker_image_files.py` is in the run because PR #123 put the background sample in the Docker image. It checks that every file the service reads at run time is in the image.
5. If the breakdown fails, the estimate still comes back with both breakdown fields null and a warning (`test_valuation_breakdown_failure.py`).